

Programmable Built-In Self-Test (PBIST) Module

This chapter describes the programmable built-in self-test (PBIST) controller module used for testing the on-chip memories on the Hercules microcontrollers.

Topic	Page
7.1 Overview	348
7.2 RAM Grouping and Algorithm	349
7.3 PBIST Flow	350
7.4 Memory Test Algorithms on the On-chip ROM	352
7.5 PBIST Control Registers	354
7.6 PBIST Configuration Example	367

7.1 Overview

The PBIST (Programmable Built-In Self-Test) controller architecture provides a run-time-programmable memory BIST engine for varying levels of coverage across many embedded memory instances.

7.1.1 Features of PBIST

- Information regarding on-chip memories, memory groupings, memory background patterns and test algorithms stored in dedicated on-chip PBIST ROM
- Host processor interface to configure and start BIST of memories
- Supports testing of PBIST ROM itself as well
- Supports testing of each memory at its maximum access speed in application
- Implements intelligent clock gating to conserve power

NOTE: Refer to the device datasheet for the maximum PBIST ROM clock frequency supported.

7.1.2 PBIST vs. Application Software-Based Testing

The PBIST architecture consists of a small coprocessor with a dedicated instruction set targeted specifically toward testing memories. This coprocessor executes test routines stored in the PBIST ROM and runs them on multiple on-chip memory instances. The on-chip memory configuration information is also stored in the PBIST ROM. The testing is done in parallel for each of the CPU data RAMs, while it is done sequentially for the rest of the memories.

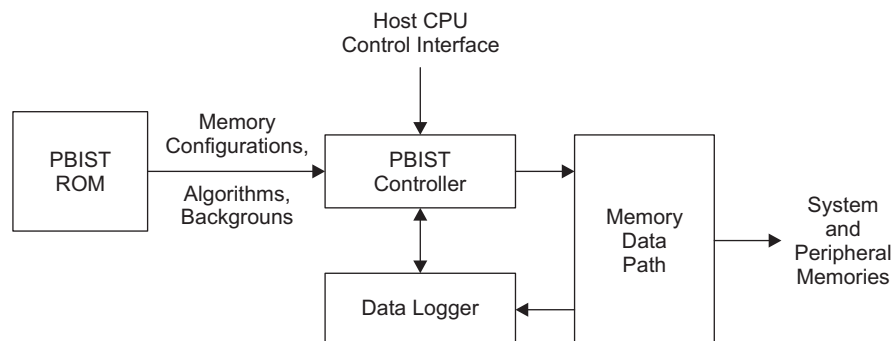
The PBIST Controller architecture offers significant advantages over tests running on the main Cortex-R4F processor (application software-based testing):

- Embedded CPUs have a long access path to memories outside the tightly-couple memory sub-system, while the PBIST controller has a dedicated path to the memories specifically for the self-test
- Embedded CPUs are designed for their targeted use and are often not easily programmed for memory test algorithms.
- The memory test algorithm code on embedded CPUs is typically significantly larger than that needed for PBIST.
- The embedded CPU is significantly larger than the PBIST controller.

7.1.3 PBIST Block Diagram

Figure 7-1 illustrates the basic PBIST blocks and its wrapper logic for the device.

Figure 7-1. PBIST Block Diagram



7.1.3.1 On-chip ROM

The on-chip ROM contains the information regarding the algorithms and memories to be tested.

7.1.3.2 Host Processor Interface to the PBIST Controller Registers

The Cortex-R4F CPU can select the algorithm and RAM groups for the memories' self-test from the on-chip ROM based on the application requirements. Once the self-test has executed, the CPU can query the PBIST controller registers to identify any memories that failed the self-test and to then take appropriate next steps as required by the application's author.

7.1.3.3 Memory Data Path

This is the read and write data path logic between different system and peripheral memories tightly coupled to the PBIST memory interface. The PBIST controller executes each selected algorithm on each valid memory group sequentially until all the algorithms are executed.

NOTE: Not all algorithms are designed to run on all RAM groups. If an algorithm is selected to run on an incompatible memory, this will result in a failure. Refer to [Table 2-5](#) and for RAM grouping and algorithm information.

7.2 RAM Grouping and Algorithm

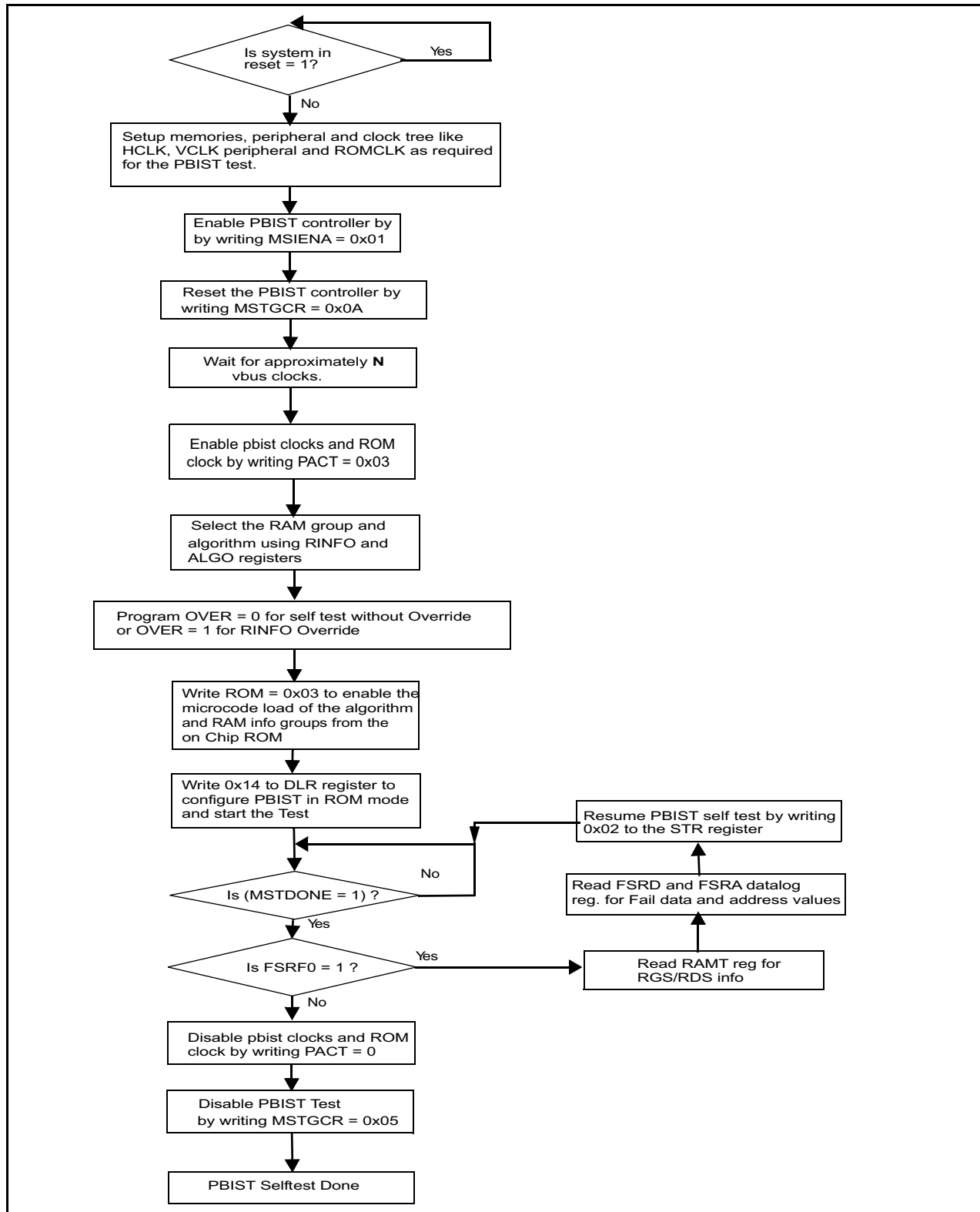
[Table 2-5](#) gives the list of RAM groups and their types supported on the device. maps the different algorithms supported in application mode for the RAM groups with the background patterns used for the particular algorithm.

NOTE: March13 is the most recommended algorithm for the memory self-test.

7.3 PBIST Flow

Figure 7-2 illustrates the memory self-test flow.

Figure 7-2. PBIST Memory Self-Test Flow Diagram



7.3.1 PBIST Sequence

1. Configure the device clock sources and domains so that they are running at their target frequencies.
2. Program the HCLK to PBIST ROM clock ratio by configuring the ROM_DIV field (bits 9:8) of the MSTGCR register of the system module. Check the device datasheet for the maximum supported PBIST ROM clock frequency.
3. Enable PBIST Controller by setting bit 1 of MSIENA register in system module.
4. Enable the PBIST self-test by writing a value of 0x0A to bits 3:0 of the MSTGCR in the system module.
5. Wait for N VBUS clock cycles based on the HCLK to PBIST ROM clock ratio:
 N = 16 when HCLK:PBIST ROM clock is 1:1
 N = 32 when HCLK:PBIST ROM clock is 1:2
 N = 64 when HCLK:PBIST ROM clock is 1:4
 N = 64 when HCLK:PBIST ROM clock is 1:8
6. Write 1h to PACT register to enable the PBIST internal clocks.
7. Program the ALGO register to decide which algorithm from the instruction ROM must be selected (the default value of ALGO register is all 1's, meaning all algorithms are selected). Similarly, program the RINFOL and RINFOU registers to indicate whether a particular RAM group in the instruction ROM would get executed or not.

NOTE: In case of RAM Override (Override Register (OVER) = 00), the user should make sure that only the algorithms that run on similar RAMs are selected. If a single port algorithm is selected in ROM Algorithm Mask Register (ALGO), the RAM Info Mask Lower Register (RINFOL) and RAM Info Mask Upper Register (RINFOU) must select only the single port RAM's. The same applies for two port RAM's. Check Architecture chapter for information on the memory types.

8. Program OVER = 1h to run PBIST self-test without RAM override. Program OVER = 0 to run PBIST self-test with RAM Override.
9. Write a value of 3h to the ROM mask register should the microcode for the Algorithms as well as the RAM groups loaded from the on-chip PBIST ROM.
10. Write DLR (Data Logger register) with 14h to configure the PBIST run in ROM mode and to enable the configuration access. This starts the memory self-tests.
11. Wait for the PBIST self-test done by polling MSTDONE bit of MSTCGSTAT register in System Module.
12. Once self-test is completed, check the Fail Status register FSRF0.
 In case there is a failure (FSRF0 = 1h):
 - a. Read RAMT register that indicates the RGS and RDS values of the failure RAM
 - b. Read FSRC0 and FSRC1 registers that contains the failure count
 - c. Read FSRA0 and FSRA1 registers that contains the address of first failure
 - d. Read FSRDL0 and FSRDL1 registers that contains the failure data.
 - e. Write a value of 2h to the STR register to resume the test.
 In case there is no failure (FSRF0 = 0) the memory self-test is completed.
 - a. Disable the PBIST internal clocks by writing a 0 to the PACT register.
 - b. Disable the PBIST self-test by writing a value of 5h to bits 3:0 of the MSTGCR in the system module.
13. Repeat steps 2 through 9 for subsequent runs with different RAM group and algorithm configurations.
14. After required Memory tests are completed, Resume or Start the Normal Application software.

NOTE: The contents of the selected memory before the test will be completely lost. User software must take care of data backup if required. Typically the PBIST tests are carried out at the beginning of Application software.

NOTE: Memory test fail information is reported in terms of RGS:RDS and not RAM GROUP. Check [Table 2-5](#) for information on the RGS:RDS information applicable to each memory being tested.

7.4 Memory Test Algorithms on the On-chip ROM

This section provides a brief description for some of the test algorithms used for memory self-test.

1. March13N:

- March13N is the baseline test algorithm for SRAM testing. It provides the highest overall coverage. The other algorithms provide additional coverage of otherwise missed boundary conditions of the SRAM operation.
- The concept behind the general march algorithm is to indicate:
 - The bit cell can be written and read as both a 1 and a 0.
 - The bits around the bit cell do not affect the bit cell.
- The basic operation of the march is to initialize the array to a know pattern, then march a different pattern through the memory.
- Type of faults detected by this algorithm:
 - Address decoder faults
 - Stuck-At faults
 - Coupled faults
 - State coupling faults
 - Parametric faults
 - Write recovery faults
 - Read/write logic faults

2. Map Column:

- The MAP COLUMN algorithm is used to identify bit line sensitivities in the memory array. The memory array is loaded with a row stripe pattern of all 1s in the first row followed by all 0s in the second row and repeated throughout the array. Then the values are read down each column on consecutive cycles. The pattern in memory is inverted and run the column reads again.
- This particular pattern is looking for the following SRAM failure mechanisms:
 - Leakage due to a low resist path in a bit
 - An Open in the bit cell
 - Leakage on a BIT or BITN line
 - Miss-balance in the sense amp
 - Leakage in the sense
 - High resist in the sense amp
 - Failure of the pre-charge circuits after read operations

3. Pre-Charge:

- The Pre-Charge algorithm exercises the pre-charge capability within the SRAM array. It is important to specifically target this issue as it is the only part of the analog portion of the SRAM that is frequency sensitive.
- Similar to the MAP COLUMN algorithm, this algorithm works its way down the columns of the SRAM. However, unlike the MAP COLUMN, this algorithm sandwiches a write between two reads to force the worst-case conditions for the pre-charge circuits in the array.
- This test will fail when an increase in system frequency nears the minimum access time of the array, at this boundary:
 - High voltage should operate better than low voltage.
 - Likewise, low temperature should operate better than high temperature.

4. DOWN1a:

- The Down1 pattern forces the switching of all data bits and most address bits on consecutive read cycles. This is primarily a read/write test of the CPU/memory subsystem.
- The aggressive writes target at-speed write failures.
- It also targets row/column decode in the memory array.
- Targets the sense amps and sense amp multiplexors.
- Memory array output buffers.
- This algorithm operates as follows:
 - Load 1st half of the memory under test with one pattern.
 - Load 2nd half of the memory under test with the bit-wise inverse of the pattern.
 - Alternate sequential reads sequences between one sequence starting at the beginning of the array and a second sequence starting at the end of the array.
 - Upon completion of the read back, invert the patterns in both halves of the array and repeat the above step.
 - Perform an aggressive write sequence by alternating writes between the bottom half of the memory upwards with a data pattern and the top half of the memory downwards with the inverse data pattern.
 - Invert the data pattern for the above two steps to perform another sequence of aggressive writes.

5. DTXN2a:

This algorithm is used to target the global column decode Logic.

7.5 PBIST Control Registers

PBIST controller uses configuration registers for programming the algorithm and its execution. All the configuration registers are memory mapped for access by the CPU through the Peripheral Bus interface. The base address for the control registers is FFFF E400h.

NOTE: There is no watchdog functionality implemented in the PBIST controller. If a bad code is executed, the PBIST runs forever. The PBIST controller does not guard against this situation.

Registers are accessible only when the clock to the PBIST controller is active. The clock is activated by first writing 1h to the PACT register.

Table 7-1. PBIST Registers

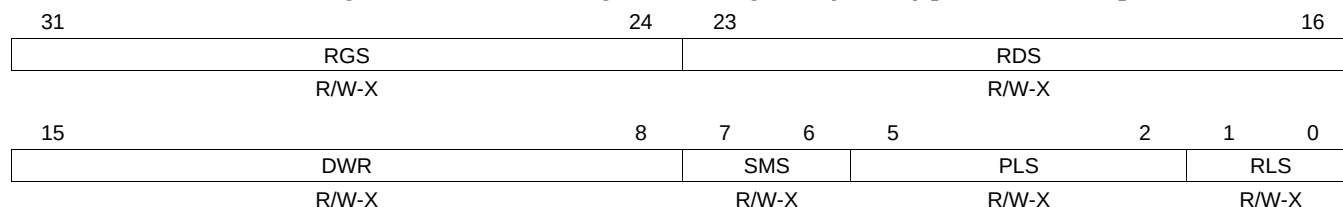
Offset	Acronym	Register Description	Section
000h - 15Ch	Reserved	Reserved locations. Do not write to these locations.	
160h	RAMT	RAM Configuration Register	Section 7.5.1
164h	DLR	Datalogger Register	Section 7.5.2
168h - 17Ch	Reserved	Reserved locations. Do not write to these locations.	
180h	PACT	PBIST Activate/Clock Enable Register	Section 7.5.3
184h	PBISTID	PBIST ID Register	Section 7.5.4
188h	OVER	Override Register	Section 7.5.5
190h	FSRF0	Fail Status Fail Register 0	Section 7.5.6
198h	FSRC0	Fail Status Count Register 0	Section 7.5.7
19Ch	FSRC1	Fail Status Count Register 1	Section 7.5.7
1A0h	FSRA0	Fail Status Address 0 Register	Section 7.5.8
1A4h	FSRA1	Fail Status Address 1 Register	Section 7.5.8
1A8h	FSRDL0	Fail Status Data Register 0	Section 7.5.9
1B0h	FSRDL1	Fail Status Data Register 1	Section 7.5.9
1C0h	ROM	ROM Mask Register	Section 7.5.10
1C4h	ALGO	ROM Algorithm Mask Register	Section 7.5.11
1C8h	RINFOL	RAM Info Mask Lower Register	Section 7.5.12
1CCh	RINFOU	RAM Info Mask Upper Register	Section 7.5.13

7.5.1 RAM Configuration Register (RAMT)

This register is divided into the following internal registers, none of which have a default value after reset. [Figure 7-3](#) and [Table 7-2](#) illustrate this register.

This register provides the information regarding the memory being currently tested. In case of a PBIST failure, the application can read this register to identify the RGS:RDS values for the memory that failed the self-test.

Figure 7-3. RAM Configuration Register (RAMT) [offset = 0160h]



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

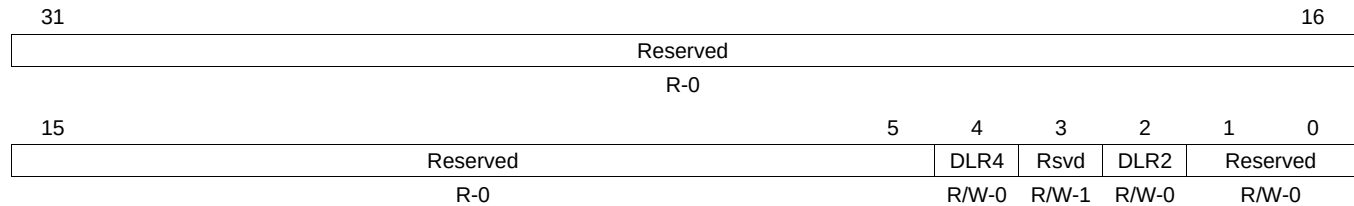
Table 7-2. RAM Configuration Register (RAMT) Field Descriptions

Bit	Field	Description
31-24	RGS	Ram Group Select. Refer to Table 2-5 for information on the RGS value for each memory.
23-16	RDS	Return Data Select. Refer to Table 2-5 for information on the RDS values for each memory.
15-8	DWR	Data Width Register
7-6	SMS	Sense Margin Select Register
5-2	PLS	Pipeline Latency Select
1-0	RLS	RAM Latency Select

7.5.2 Datalogger Register (DLR)

This register puts the PBIST controller into the appropriate comparison modes for data logging. [Figure 7-4](#) and [Table 7-3](#) illustrate this register.

Figure 7-4. Datalogger Register (DLR) [offset = 0164h]



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 7-3. Datalogger Register (DLR) Field Descriptions

Bit	Field	Value	Description
31-5	Reserved	0	Reads return 0. Do not change these bits from their default value.
4	DLR4		Config access: setting this bit allows the host processor to configure the PBIST controller registers.
3	Reserved	1	Do not change this bit from its default value of 1.
2	DLR2		ROM-based testing: setting this bit enables the PBIST controller to execute test algorithms that are stored in the PBIST ROM.
1-0	Reserved	00	Do not change these bits from their default value of 00.

- DLR2: ROM-based testing mode**

Writing a 1 to this register starts the ROM-based testing. This register is used to initiate ROM-based testing from Config and ATE interfaces. Also, since a 1 in this bit position means the instruction ROM is used for memory testing, all the intermediate interrupts and PBIST done signal after each memory test are masked until all the selected algorithms in the ROM are executed for all RAM groups. However, a failure would stop the test and report the status immediately.

- DLR4: Config access mode**

This mode, when set, indicates the CPU is being used to access PBIST.

This is the first register that needs to be programmed to activate the PBIST controller. Bit [0] is used for static clock gating, and unless a 1 is written to this bit, all the internal PBIST clocks are shut off. [Figure 7-5](#) and [Table 7-4](#) illustrate this register.

NOTE: This register must be programmed to 1h during application self-test.

31			16
Reserved			
R-0			
15			0
Reserved			PACT0
R-0			R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

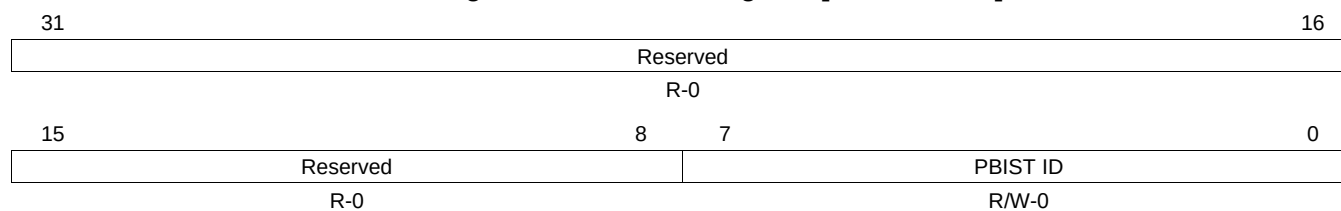
Bit	Field	Value	Description
31-1	Reserved	0	Reads return 0. Writes have no effect.
0	PACT0	0	PBIST internal clocks enable.
		0	Disable PBIST internal clocks.
		1	Enable PBIST internal clocks.

This bit must be set to 1 to turn on the PBISt internal clocks. Setting this bit asserts an internal signal that is used as the clock gate enable. As long as this bit is 0, any access to the PBISt will not go through and the PBISt will remain in an almost zero-power mode.

7.5.4 PBIST ID Register

Functionality of this register is described in [Figure 7-6](#) and [Table 7-5](#).

Figure 7-6. PBIST ID Register [offset = 184h]



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 7-5. PBIST ID Register Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	PBIST ID		This is a unique ID assigned to each PBIST controller in a device with multiple PBIST controllers.

7.5.5 Override Register (OVER)

Functionality of the register is described in [Figure 7-7](#) and [Table 7-6](#).

Figure 7-7. Override Register (OVER) [offset = 0188h]

31																	16
Reserved																	
R-0																	
15													3	2	1	0	
Reserved												Reserved		OVER0			
R-0												R-0		R-0	R/W-1		

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 7-6. Override Register (OVER) Field Descriptions

Bit	Field	Value	Description
31-3	Reserved	0	Reads return 0. Writes have no effect.
2	Reserved	0	Reserved. This bit must not be changed from its default value of 0.
1	Reserved	0	Reserved. This bit must not be changed from its default value of 0.
0	OVER0	0	RINFO Override Bit The RAM info registers RINFOL and RINFOU are used to select the memories for test.
		1	The memory information available from ROM will override the RAM selection from the RAM info registers RINFOL and RINFOU.

• OVER0

While doing ROM-based testing, each algorithm downloaded from the ROM has a memory mask associated with it that defines the applicable memory groups the algorithm will be run on. By default, this bit is set to 1, which means the memory mask that is downloaded from the ROM will overwrite the RAM info registers. The override bit can be reset by writing a 0 to it. In this case, the application can select the RAM groups to be tested by configuring the RAM info registers.

NOTE: When this override bit = 0, each algorithm selected in ALGO register will run on each RAM selected in RINFOL and RINFOU register. It must be ensured that:

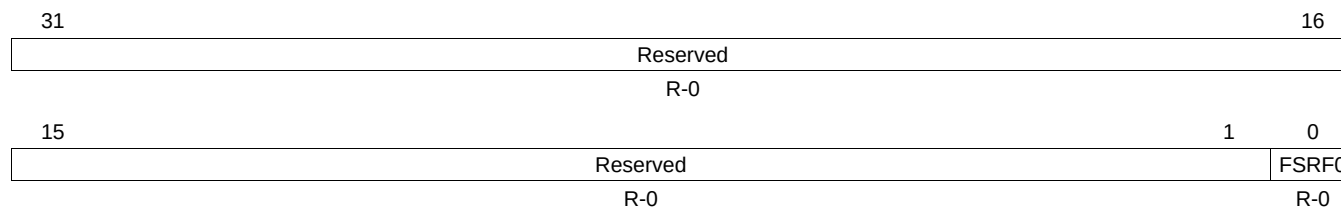
1. Only the same type of memories (single port or two port) are selected, and
2. Only memories that are valid for all algorithms enabled via the ALGO register are selected.

If the above two requirements are not met, the memory self-test will fail.

7.5.6 Fail Status Fail Register (FSRF0)

This register indicates if a Port0 failure occurred during a memory self-test. Bit [0] gets set whenever a failure occurs. Functionality of the register is described in [Figure 7-8](#) and [Table 7-7](#).

Figure 7-8. Fail Status Fail Register 0 (FSRF0) [offset = 0190h]



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

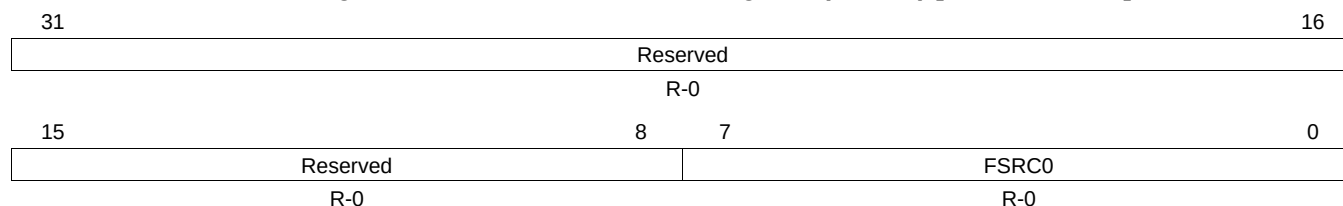
Table 7-7. Fail Status Fail Register 0 (FSRF0) Field Descriptions

Bit	Field	Value	Description
31-1	Reserved	0	Reads return 0. Writes have no effect.
0	FSRF0	0	Fail Status 0. This bit would be cleared by reset of the module using MSTGCR register in system module.
		1	No failure occurred.
		1	Failure occurred on port 0.

7.5.7 Fail Status Count Registers (FSRC0 and FSRC1)

These registers keep count of the number of failures observed during the memory self-test. The PBIST controller stops executing the memory self-test whenever a failure occurs in any memory instance for any of the test algorithms. The value in FSRC0 / FSRC1 gets incremented by one whenever a failure occurs and gets decremented by one when the failure is processed. FSRC0 is for Port 0 and FSRC1 is for Port 1. [Figure 7-9](#) and [Table 7-8](#) illustrate the FSRC0 register, while [Figure 7-10](#) and [Table 7-9](#) illustrate the FSRC1 register.

Figure 7-9. Fail Status Count 0 Register (FSRC0) [offset = 0198h]

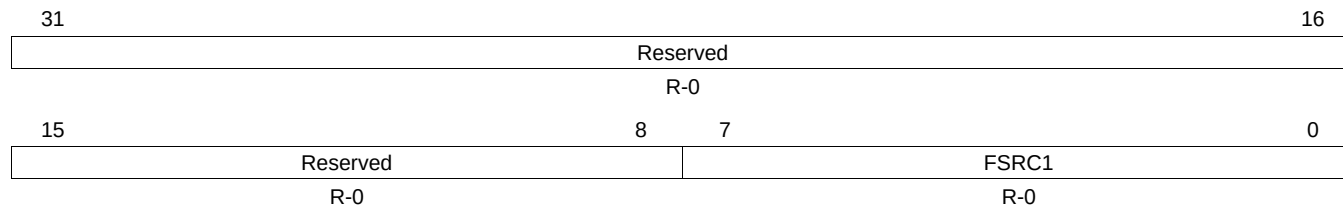


LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 7-8. Fail Status Count 0 Register (FSRC0) Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	FSRC0		Fail Status Count 0. Indicates the number of failures on port 0.

Figure 7-10. Fail Status Count Register 1 (FSRC1) [offset = 019Ch]



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

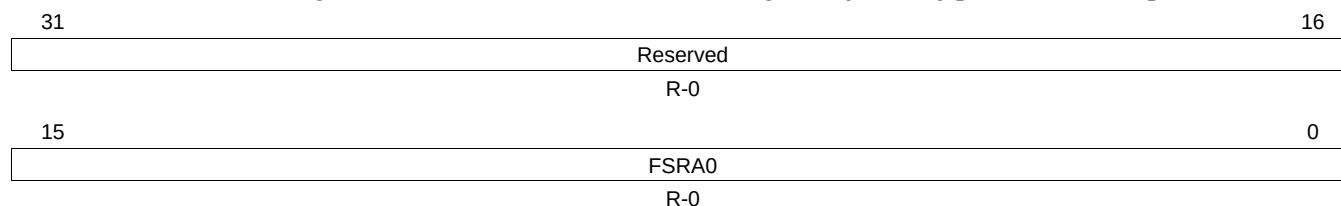
Table 7-9. Fail Status Count Register 1 (FSRC1) Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	FSRC1		Fail Status Count 1. Indicates the number of failures on port 1.

7.5.8 Fail Status Address Registers (FSRA0 and FSRA1)

These registers capture the memory address of the first failure on port 0 and port 1, respectively. [Figure 7-11](#) and [Table 7-10](#) illustrate the FSRA0 register, while [Figure 7-12](#) and [Table 7-11](#) illustrate the FSRA1 register.

Figure 7-11. Fail Status Address 0 Register (FSRA0) [offset = 01A0h]

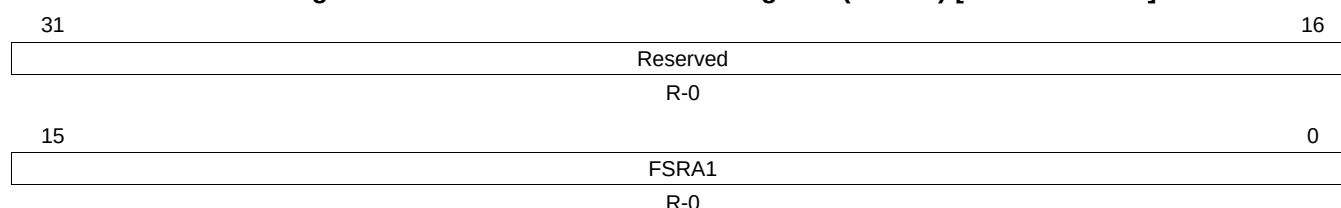


LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 7-10. Fail Status Address 0 Register (FSRA0) Field Descriptions

Bit	Field	Value	Description
31-16	Reserved	0	Reads return 0. Writes have no effect.
15-0	FSRA0		Fail Status Address 0. Contains the address of the first failure.

Figure 7-12. Fail Status Address 1 Register (FSRA1) [offset = 01A4h]



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

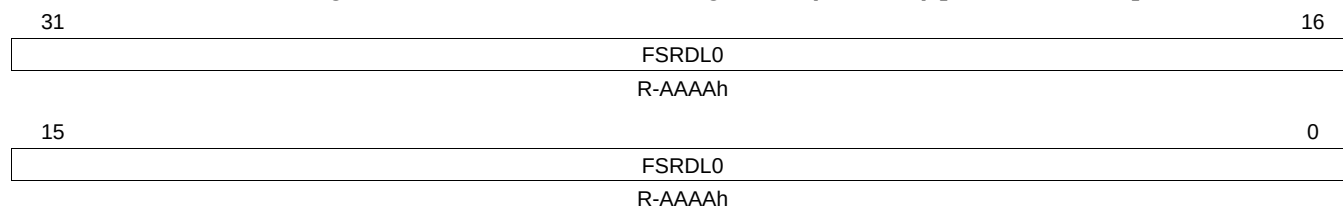
Table 7-11. Fail Status Address 1 Register (FSRA1) Field Descriptions

Bit	Field	Value	Description
31-16	Reserved	0	Reads return 0. Writes have no effect.
15-0	FSRA1		Fail Status Address 1. Contains the address of the first failure.

7.5.9 Fail Status Data Registers (FSRDL0 and FSRDL1)

These registers are used to capture the failure data in case of a memory self-test failure. FSRDL0 corresponds to Port 0, while FSRDL1 corresponds to Port 1. [Figure 7-13](#) and [Table 7-12](#) illustrate the FSRDL0 register, while [Figure 7-14](#) and [Table 7-13](#) illustrate the FSRDL1 register.

Figure 7-13. Fail Status Data Register 0 (FSRDL0) [offset = 01A8h]

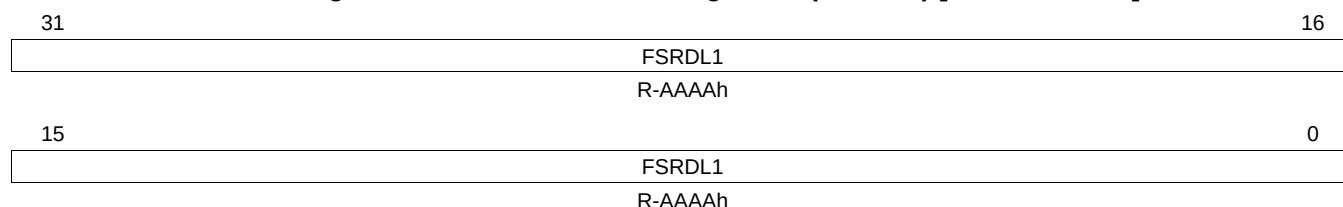


LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 7-12. Fail Status Data Register 0 (FSRDL0) Field Descriptions

Bit	Field	Description
31-0	FSRDL0	Failure data on port 0

Figure 7-14. Fail Status Data Register 1 (FSRDL1) [offset = 01B0h]



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

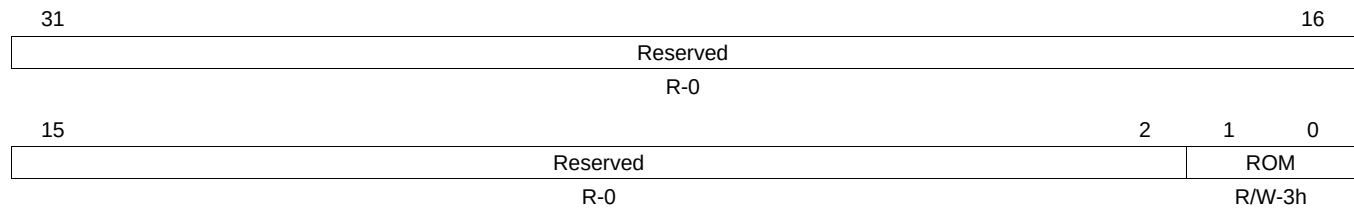
Table 7-13. Fail Status Data Register 1 (FSRDL1) Field Descriptions

Bit	Field	Description
31-0	FSRDL1	Failure data on port 1

7.5.10 ROM Mask Register (ROM)

This two-bit register sets appropriate ROM access modes for the PBIST controller. The default value is 11b. This register is illustrated in [Figure 7-15](#). It can be programmed according to [Table 7-14](#).

Figure 7-15. ROM Mask Register (ROM) [offset = 01C0h]



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

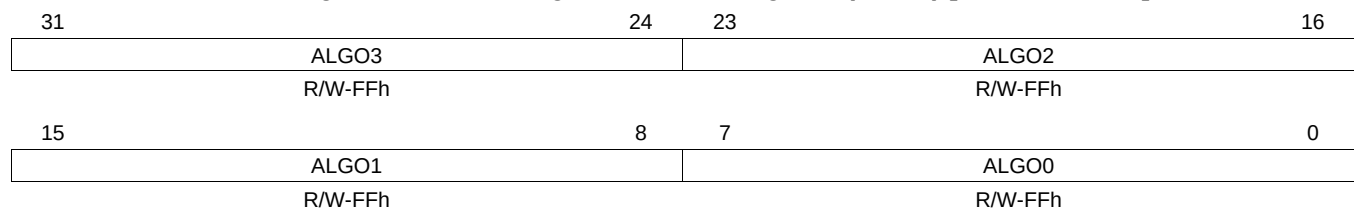
Table 7-14. ROM Mask Register (ROM) Field Descriptions

Bit	Field	Value	Description
31-2	Reserved	0	Reads return 0. Writes have no effect.
1-0	ROM	0	No information is used from ROM.
		1h	Only RAM Group information from ROM.
		2h	Only Algorithm information from ROM.
		3h	Both Algorithm and RAM Group information from ROM. This option should be selected for application self-test.

7.5.11 ROM Algorithm Mask Register (ALGO)

This register is used to indicate the algorithm(s) to be used for the memory self-test routine. Each bit corresponds to a specific algorithm. For example, bit [0] controls whether algorithm 1 is enabled or not. [Figure 7-16](#) and [Table 7-15](#) illustrate this register.

Figure 7-16. ROM Algorithm Mask Register (ALGO) [offset = 01C4h]



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 7-15. Algorithm Mask Register (ALGO) Field Descriptions

Bit	Field	Value	Description
31		0	Algorithm 32 is not selected.
		1	Selects algorithm 32 for PBIST run.
30		0	Algorithm 31 is not selected.
		1	Selects algorithm 31 for PBIST run.
:			
0		0	Algorithm 1 is not selected.
		1	Selects algorithm 1 for PBIST run.
31-0		0	None of the algorithms are selected.

NOTE: Please refer to for available algorithms and the memories on which each algorithm can be run.

7.5.12 RAM Info Mask Lower Register (RINFOL)

This register is to select RAM groups to run the algorithms selected in the ALGO register. For an algorithm to be executed on a particular RAM group, the corresponding bit in this register must be set to 1. The default value of this register is all 1s, which means all the RAM Groups are selected. [Figure 7-17](#) and [Table 7-16](#) illustrate this register.

The information from this register is used only when bit 0 in OVER register is not set.

Figure 7-17. RAM Info Mask Lower Register (RINFOL) [offset = 01C8h]

31	24	23	16
RINFOL3		RINFOL2	
R/W-FFh		R/W-FFh	
15	8	7	0
RINFOL1		RINFOL0	
R/W-FFh		R/W-FFh	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 7-16. RAM Info Mask Lower Register (RINFOL) Field Descriptions

Bit	Field	Value	Description
31		0	RAM Group 32 is not selected.
		1	Selects group 32 for PBIST run.
30		0	RAM Group 31 is not selected.
		1	Selects RAM group 31 for PBIST run.
:			
0		0	RAM Group 1 is not selected.
		1	Selects RAM Group 1 for PBIST run.
31-0		0	None of the RAM Groups 1 to 32 are selected.

NOTE: Please refer to [Table 2-5](#) for RAM info groups.

7.5.13 RAM Info Mask Upper Register (RINFOU)

This register is to select RAM groups to run the algorithms selected in the ALGO register. For an algorithm to be executed on a particular RAM group, the corresponding bit in this register should be set to 1. The default value of this register is all 1s, which means all the RAM Info Groups would be selected. [Figure 7-18](#) and [Table 7-17](#) illustrate this register.

Figure 7-18. RAM Info Mask Upper Register (RINFOU) [offset = 01CCh]

31	24	23	16
RINFOU3		RINFOU2	
R/W-FFh		R/W-FFh	
15	8	7	0
RINFOU1		RINFOU0	
R/W-FFh		R/W-FFh	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 7-17. RAM Info Mask Upper Register (RINFOU) Field Descriptions

Bit	Field	Value	Description
31		0	RAM Group 64 is not selected.
		1	Selects group 64 for PBIST run.
30		0	RAM Group 63 is not selected.
		1	Selects RAM group 63 for PBIST run.
:			
0		0	RAM Group 33 is not selected.
		1	Selects RAM Group 33 for PBIST run.
31-0		0	None of RAM Groups 33 to 64 are selected.

7.6 PBIST Configuration Example

The following examples assume that the PLL is locked and selected as clock source with HCLK = 160 MHz and VCLK = 80 MHz.

7.6.1 Example 1 : Configuration of PBIST Controller to Run Self-Test on RAM Group 3

This example explains the configurations for running March13, Down1A and Map Column algorithms on RAM Group 3 (see device datasheet for RAM Group information).

1. Program the HCLK to PBIST ROM clock ratio to 1:2 in System Module.
MSTGCR[9:8] = 1
2. Enable PBIST Controller in System Module.
MSIENA[31:0] = 0x00000001
3. Enable the PBIST self-test in System Module.
MSTGCR[3:0] = 0xA
4. Wait for at least 32 VCLK cycles in a software loop.
5. Enable the PBIST internal clocks.
PACT = 0x1
6. Disable RAM Override. This will make the PBIST controller use the information provided by the application in the RINFOx and ALGO registers for the memory self-test.
OVER = 0x0
7. Select the Algorithm (refer to).
ALGO = 0x00000054 (Algo 3 = March13N, Algo 5 = down1A_red, Algo 7 = Map column for two-port RAM Group 3)
8. Program the RAM group Info to select RAM Group 3 (refer to [Table 2-5](#)).
RINFOL = 0x00000004 (select RAM Group 3)
RINFOU = 0x00000000 (since this device supports only 28 RAM Groups)
9. Select both Algorithm and RAM information from on-chip PBIST ROM.
ROM = 0x3
10. Configure PBIST to run in ROM Mode and start PBIST run.
DLR = 0x14
11. Wait for PBIST test to complete by polling MSTDONE bit in System Module.
while (MSTDONE !=1)
12. Once self-test is completed, check the Fail Status register FSRF0.
 - a. In case there is a failure (FSRF0 = 0x01):
 - i. Read RAMT register that indicates the RGS and RDS values of the failure RAM.
 - ii. Read FSRC0 and FSRC1 registers that contains the failure count.
 - iii. Read FSRA0 and FSRA1 registers that contains the address of first failure.
 - iv. Read FSRDL0 and FSRDL1 registers that contains the failure data.
 - v. Resume the Test if required using Program Control register (offset = 0x16C) STR = 2.
 - b. In case there is no failure (FSRF0 = 0x00), the memory self-test is completed:
 - i. Disable the PBIST internal clocks.
PACT = 0
 - ii. Disable the PBIST self-test.
MSTGCR[3:0] = 0x5

7.6.2 Example 2 : Configuration of PBIST Controller to Run Self-Test on ALL RAM Groups

This example explains the configurations for running March13, Down1A and Mapcolumn algorithms on all RAM groups defined in the PBIST ROM.

1. Program the HCLK to PBIST ROM clock ratio to 1:2 in System Module.
MSTGCR[9:8] = 1
2. Enable PBIST Controller in System Module.
MSIENA[31:0] = 0x00000001
3. Enable the PBIST self-test in System Module.
MSTGCR[3:0] = 0xA
4. Wait for at least 32 VCLK cycles in a software loop.
5. Enable the PBIST internal clocks.
PACT = 0x1
6. Enable RAM Override.
OVER = 0x1
7. Select the Algorithms to be run (refer to).
ALGO = 0x000000FC (select March13N, Down1A and Map Column algorithms for single-port and two-port RAMs)
8. Select both Algorithm and RAM information from on-chip PBIST ROM.
ROM = 0x3
9. Configure PBIST to run in ROM Mode and kickoff PBIST test.
DLR = 0x14
10. Wait for PBIST test to complete by polling MSTDONE bit in System Module.
while (MSTDONE !=1)
11. Once self-test is completed, check the Fail Status register FSRF0:
 - a. In case there is a failure (FSRF0 = 0x01):
 - i. Read RAMT register that indicates the RGS and RDS values of the failure RAM.
 - ii. Read FSRC0 and FSRC1 registers that contains the failure count.
 - iii. Read FSRA0 and FSRA1 registers that contains the address of first failure.
 - iv. Read FSRDL0 and FSRDL1 registers that contains the failure data.
 - v. Resume the Test if required using Program Control register (offset = 0x16C) STR = 2.
 - b. In case there is no failure (FSRF0 = 0x00), the memory self-test is completed:
 - i. Disable the PBIST internal clocks.
PACT = 0
 - ii. Disable the PBIST self-test.
MSTGCR[3:0] = 0x5